

NAAN MUDHALVAN PHASE-WISE PROJECT REPORT

ServiceNow Incident Management – Client Script & UI Policy

Implementing Client Script and UI Policy on the Incident form

Prepared from the phase-wise templates and project materials uploaded for this report.

Project demonstration supplied by the Team

REPORT NOTE

Scope: This report consolidates the uploaded Naan Mudhalvan templates and adapts them to the demonstrated ServiceNow Incident project. Generic template examples are not presented as actual project evidence. Team IDs, names, screenshots, credentials and measured test results that were not supplied are not fabricated.

Project identification: The supplied YouTube demonstration is titled “Implement Client Script & UI Policy (Incident)”, so the project focus is ServiceNow Incident form customization using Client Script and UI Policy.

Source material: Ideation, planning, problem–solution fit, proposed solution, solution architecture, requirements, DFD/user stories, technology stack, final report/FSD formats, performance-testing templates and UAT template.

TABLE OF CONTENTS

Section	Topic
1	Introduction
2	Ideation Phase
3	Requirement Analysis
4	Project Design
5	Project Planning & Scheduling
6	Implementation
7	Functional & Performance Testing
8	Results
9	Advantages & Disadvantages
10	Conclusion
11	Future Scope
12	Appendix

1. INTRODUCTION

1.1 Project Overview

ServiceNow Incident Management is used to record, track and manage incidents reported by users. This project customizes the Incident form so that user interaction is controlled through client-side logic and UI policies. The solution helps guide users, validate information and control field behaviour.

1.2 Purpose

- Demonstrate practical ServiceNow Incident form customization.
- Reduce incorrect or incomplete Incident data through form-level controls.
- Improve consistency of Incident records through UI behaviour.
- Follow the Naan Mudhalvan workflow from ideation through testing.

1.3 Objectives

- Identify a practical Incident-management form problem.
- Define user needs and system requirements.
- Design a ServiceNow solution using Client Script and UI Policy.
- Plan work using Agile concepts such as epics, user stories, sprints and story points.
- Define functional and UAT checks for the configured form.

2. IDEATION PHASE

2.1 Empathize & Discover

The uploaded Empathy Map Canvas is intended to understand user behaviour, goals and challenges. For this Incident project, the primary users are the person creating/updating an Incident and the service-desk person who needs reliable Incident information.

Empathy area	Project-specific interpretation
Says	The user wants to report an issue quickly and expects the form to guide them.
Thinks	The user needs to know which information is required and which fields can be changed.
Does	Creates an Incident, enters details, selects values and submits or updates the record.
Feels	May become frustrated when required information is unclear or invalid data is accepted.
Pain points	Missing information, inconsistent values, unnecessary editable fields and avoidable data-entry errors.
Needs	Clear form behaviour, validation, appropriate mandatory fields and controlled editability.

2.2 Problem Statement

Problem statement: Service-desk users need a controlled and understandable Incident form because incomplete or inconsistent Incident information can make tracking and resolution harder. The system should guide users at form level so required information is captured and inappropriate field interaction is reduced.

2.3 Brainstorming & Idea Generation

- Use Client Scripts to validate or react to user input.
- Use UI Policies to control fields as mandatory, visible/hidden or read-only.
- Provide clearer Incident-form guidance.
- Reduce unnecessary manual checks by applying rules directly on the form.
- Keep declarative UI controls separate from broader server-side processing.

2.4 Idea Prioritization

Idea	Value	Effort	Priority rationale
UI Policy for field control	High	Low	Directly addresses form usability and data completeness
Client Script validation	High	Medium	Supports dynamic validation and user feedback.

3. REQUIREMENT ANALYSIS

3.1 Customer / User Journey

Stage	User action	System expectation
Identify issue	Decides to report an incident.	Incident form is available.
Create incident	Opens the Incident form.	Relevant fields are presented clearly.
Enter information	Enters description, category, priority and other details.	Rules guide the user and control invalid/incomplete input.
Submit/update	Saves the record.	Incident is stored with consistent information.
Track/resolve	Service desk updates the Incident.	Controlled fields support consistent tracking.

3.2 Functional Requirements

FR ID	Functional requirement	Expected behaviour
FR-01	Open Incident form	User can access the Incident form.
FR-02	Capture Incident details	Form accepts required Incident information.
FR-03	Client-side validation	Client Script validates or reacts to input according to its condition.
FR-04	Dynamic field control	UI Policy makes fields mandatory, visible/hidden or read-only according to a condition.
FR-05	Save/update Incident	Valid Incident information can be saved or updated.

3.3 Non-Functional Requirements

NFR ID	Requirement	Description
NFR-01	Usability	Rules should make the form easier to understand.
NFR-02	Security	Only authorized ServiceNow users should access/modify Incident records.
NFR-03	Reliability	Configured rules should behave consistently.
NFR-04	Performance	Client-side behaviour should respond without unnecessary delay.
NFR-05	Availability	The ServiceNow environment should be available during intended use.
NFR-06	Maintainability	Rules should be clearly named and documented.

3.4 User Stories

USN	User story	Acceptance criteria	Priority
USN-1	As a service-desk user, I can create an Incident with required information	Required information is captured before submission	High
USN-2	As a user, I want the form to react to my input.	Configured client rule executes when its condition is met	High

3.5Data Flow Diagram

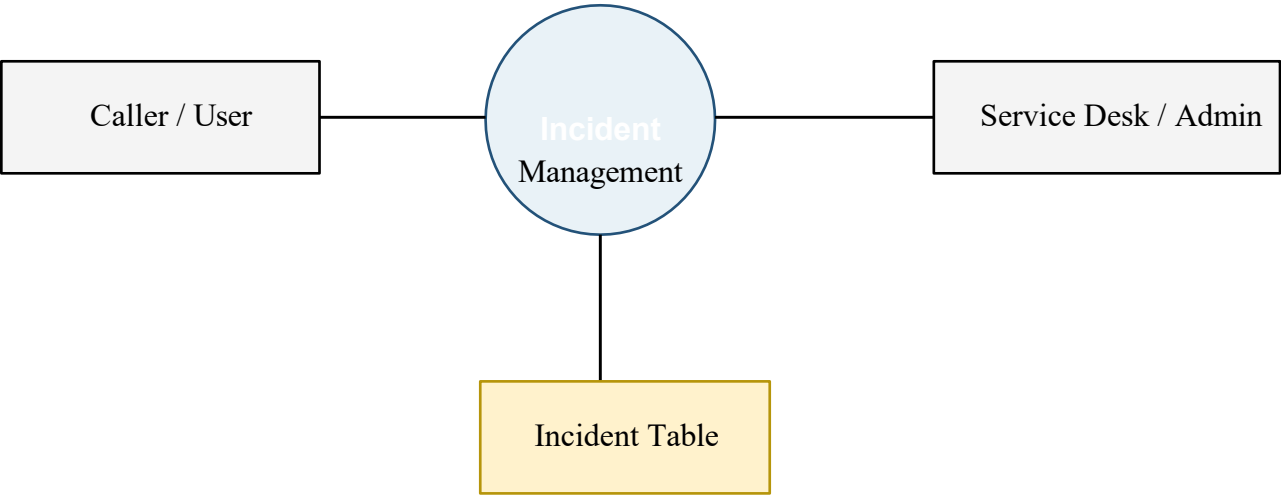


Figure 1. Simplified Incident data flow and rule application.

4. PROJECT DESIGN

4.1 Problem–Solution Fit

Problem	Solution	Fit
Incident data may be incomplete or inconsistent	.UI Policies enforce appropriate field states and Client Scripts	
Users may not know which fields are required.	Conditional mandator/read-only behaviour provides	
Manual checking creates avoidable effort.	Form rules provide immediate client-side feedback.	Improves consistency and user experie

4.2 Proposed Solution

Parameter	Project-specific description
Problem to be solved	Improve correctness and usability of Incident form data entry.
Idea / Solution	Configure ServiceNow Client Script and UI Policy rules on the Incident form.
Novelty / Uniqueness	Combines dynamic client-side behaviour with declarative UI controls.
Social / Customer impact	Helps service-desk users submit clearer and more complete Incident information.
Business value	Reduces avoidable data-entry issues and supports standardized Incident handling.
Scalability	Additional fields and conditions can be supported by adding well-defined policies/scripts.

4.3 Solution Architecture

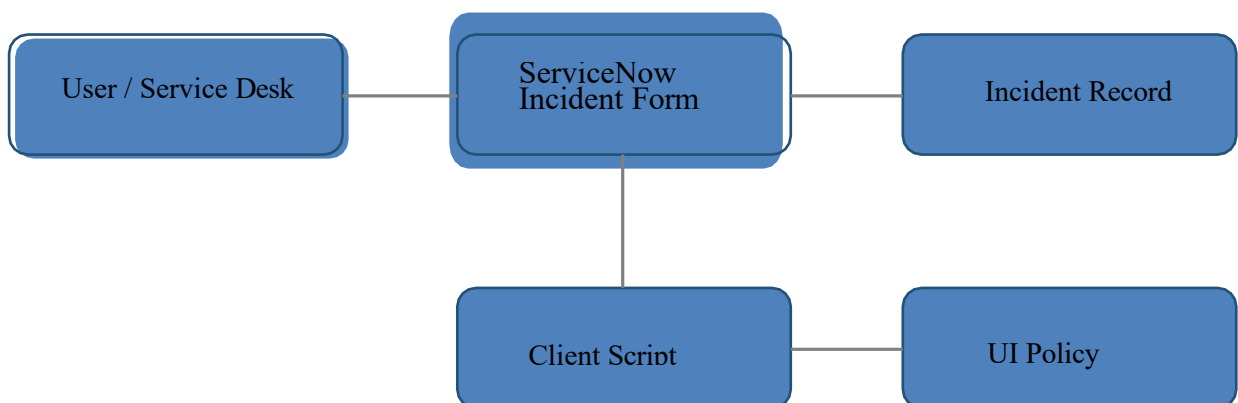


Figure 2. Project architecture

4.4 Application Flow

Step	Flow
1	User opens/updates Incident form.
2	User enters or changes Incident information.
3	UI Policy evaluates its condition and changes configured field states.
4	Client Script executes for its configured form event/condition.
5	Valid information is saved to the Incident record.

5. PROJECT PLANNING & SCHEDULING

5.1 Submitted Agile Planning Evidence

The uploaded planning logic records 12 story points for Sprint 1 and 20 for Sprint 2, giving 32 total story points. With two sprints, the submitted calculation is a velocity of 16 story points per sprint. The original planning document uses a generic data/visualization example, so that evidence is kept separate from the adapted ServiceNow backlog below.

Submitted planning evidence	Value
Sprint 1	12 story points
Sprint 2	20 story points
Total	32 story points
Number of sprints	2
Velocity	16 story points per sprint

5.2 Adapted Project Backlog

Epic	Story / Task	Points	Priority	Sprint
Ideation	Define Incident-management problem	2	High	1
Requirements	Identify functional and non-functional requirements	3	High	1
Design	Prepare solution architecture and DFD	3	High	1
Implementation	Configure Incident form UI Policy	3	High	1
Implementation	Configure Client Script	5	High	2
Testing	Functional test cases and fixes	3	High	2
Documentation	Prepare FSD and final report	3	Medium	2
Demo	Prepare project demonstration	2	Medium	2

5.3 Sprint Schedule

Sprint	Focus	Planned outcome
Sprint 1	Ideation, requirements, design, UI Policy	Problem definition, requirements, architecture and core UI control
Sprint 2	Client Script, testing, documentation and demo	Dynamic validation, testing evidence, documentation and demons

5.4 Burndown

A burndown chart shows work remaining against time. The uploaded template explains the concept but does not contain project-specific daily measurements, so this report does not fabricate daily burn values. Actual sprint/day values can be inserted before final submission.

6 IMPLEMENTATIONS

6.1 ServiceNow Incident Form

The implementation is centered on the Incident form. The supplied demonstration is specifically about implementing Client Script and UI Policy behaviour for an Incident. The two mechanisms serve complementary purposes.

Component	Purpose	Project use
UI Policy	Declaratively controls form fields when a condition is	tacked fields mandatory, read-only or visible/hidden according to Inc
Client Script	Runs client-side JavaScript for configured form event	validate or dynamically respond to user input.
Incident form	User interface for entering/updating an Incident.	Provides fields and context for the rules.
Incident record	Persisted Incident data.	Stores the resulting information for tracking and resolution.

6.2 Implementation Workflow

- Open the Incident form in the ServiceNow environment.
- Identify the field(s) and condition requiring controlled behaviour.
- Create/configure the UI Policy for declarative field control.
- Create/configure the Client Script for dynamic client-side logic where needed.
- Save the configuration and test the condition with representative Incident values.
- Confirm that form behaviour matches the intended requirement.
- Document rule names, conditions and expected outcomes.

6.3 Implementation Logic

Condition	Action	Expected user experience
Condition false	Normal field state	User interacts with the form normally.
Condition true	UI Policy applies configured field state	Relevant field becomes mandatory/read-only/hid
Triggering value changes	Client Script evaluates the change where configured	Immediate client-side response or validation occ
Valid data saved	Form submits	Incident record is stored.
Invalid/incomplete input	Configured validation flags or prevents continuation	User corrects the input.

7 FUNCTIONAL & PERFORMANCE TESTING

7.1 Functional Testing

TC ID	Test scenario	Expected result	Evidence status
TC-001	Open Incident form	Incident form loads successfully.	Demonstration scope
TC-002	Enter valid Incident information	Valid values are accepted.	Verify in instance
TC-003	Trigger UI Policy condition	Configured field state changes correctly.	Demonstration scope
TC-004	Trigger Client Script condition	Client-side logic executes correctly.	Demonstration scope
TC-005	Enter incomplete required information	Required field is identified and/or submission is controlled.	Verify in instance
TC-006	Change triggering field back	Fields return to the correct normal state.	Verify in instance
TC-007	Save valid Incident	Incident is created/updated successfully.	Verify in instance
TC-008	Update existing Incident	Configured rules continue to behave correctly.	Verify in instance

7.2 Performance Testing

The uploaded performance-testing package contains generic templates for several technologies. For this ServiceNow project, relevant performance checks are form response, UI Policy response and Client Script response. No numerical measurements were supplied, so measured values are not fabricated.

PT ID	Performance check	Suggested acceptance criterion
PT-01	Incident form load	Loads within an acceptable time in the training environment.
PT-02	UI Policy response	Field-state change occurs promptly.
PT-03	Client Script response	Validation/response occurs without noticeable delay.
PT-04	Repeated interaction	Rules remain stable during repeated create/update operation

7.3 User Acceptance Testing

The supplied UAT template requires actual results, bug tracking, tester details and sign-off. Those details were not provided, so the following UAT matrix is prepared for completion in the actual ServiceNow environment.

UAT ID	Scenario	Expected outcome	Status
UAT-01	Create Incident with valid data	Incident submits successfully.	Pending sign-off
UAT-02	Trigger UI Policy condition	Fields change according to policy.	Pending sign-off
UAT-03	Trigger Client Script	Configured logic works correctly.	Pending sign-off
UAT-04	Correct invalid input	User can correct data and continue.	Pending sign-off

8 RESULTS

8.1 Demonstrated / Expected Result

- The Incident form can be customized with Client Script and UI Policy.
- Conditional field behaviour can be applied at the user-interface level.
- The form can guide users toward complete and consistent Incident information.
- The project demonstrates practical ServiceNow configuration and client-side scripting.
- The solution can be extended with additional Incident rules, validation and automation.

9 ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages / limitations
Improves Incident-form usability.	Client-side rules depend on correct configuration.
Supports consistent data entry.	Too many rules can make a form complex.
Provides immediate user feedback.	Client-side validation is not a replacement for security controls.
Reduces avoidable manual checking.	Large/poorly designed scripts may affect responsiveness.
Easy to extend with additional rules.	Testing is required whenever fields or conditions change.

9.1 Security Consideration

Client Scripts and UI Policies primarily influence the user interface. They should not be treated as a replacement for server-side security controls, access controls or server-side validation. Production deployments should apply appropriate platform security controls separately.

10 CONCLUSION

The Naan Mudhalvan project demonstrates a complete problem-to-solution workflow around ServiceNow Incident Management. The ideation phase identifies the user problem; requirement analysis converts it into functional and non-functional requirements; design defines the architecture and interaction; Agile planning organizes the work; implementation applies Client Script and UI Policy to the Incident form; and testing defines how the configured behaviour can be verified.

The uploaded planning evidence records 32 story points and a calculated velocity of 16 story points per sprint. The project-specific backlog in this report adapts the planning method to the ServiceNow scope while keeping the submitted planning evidence separate from invented measurements.

The project provides practical exposure to ServiceNow configuration, form behaviour, client-side scripting, requirement analysis, Agile planning, documentation and testing.

11 FUTURE SCOPE

- Add more Incident form policies for category, subcategory, assignment group and priority combinations.
- Introduce additional Client Scripts for advanced field validation and guidance.
- Add notifications and workflow automation for Incident state changes.
- Add dashboards and reports for Incident trends, priority and resolution time.
- Add stronger server-side validation and access controls for production deployment.
- Introduce automated regression testing for configuration changes.
- Document reusable configuration standards for future ServiceNow applications.

12 APPENDIX

A. Uploaded Source Material Consolidated

Area	Material
Ideation	Empathy Map; Problem Statements; Brainstorming / Idea Prioritization.
Planning	Project Planning Template; Planning logic.
Design	Problem–Solution Fit; Proposed Solution; Solution Architecture.
Requirements	Solution Requirements; DFD and User Stories; Technology Stack.
Reporting	Final Report Template; FSD Documentation Format.
Testing	Functional/Performance Testing templates; UAT template.
Project reference	Student-supplied YouTube demonstration: Implement Client Script & UI Policy (Incident).

B. Final Submission Fields to Complete

- Student name(s) and team ID.
- College / department / batch details if required.
- Actual ServiceNow screenshots.
- Actual test results and Pass/Fail values.
- Tester name, date and UAT sign-off.
- Actual project/demo URL if required.
- Source code/configuration export if supplied.

C. Suggested Final Submission Order

Cover Page → Introduction → Ideation → Requirement Analysis → Project Design → Planning & Scheduling →

Implementation → Testing → Results/Screenshots → Advantages & Disadvantages → Conclusion → Future Scope

→ Appendix.

Project Demo Link: https://drive.google.com/file/d/1hTul6jo_Weyynf-S3-442fkkJAEhS_rp/view?usp=drivesdk

GitHub Link: <https://github.com/padmasrim071-bot/Implement-Client-Script-UI-policy-Incident->

